

Chương 1.4
NGÔN NGỮ JAVASCRIPT

GV BIÊN SOẠN: ThS. TRẦN KIM HƯƠNG – KHOA CNS&KT

tkhuong@dthu.edu.vn

NỘI DUNG

- **Ngôn ngữ Web cơ bản**
  - HTML5
  - CSS3
- **Ngôn ngữ lập trình Front-end**
  - JavaScript
  - TypeScript
- **Framework Front-end**
  - BootStrap
  - Jquery
  - Angular

NỘI DUNG

- Giới thiệu JavaScript
- Nhúng JavaScript vào HTML
- Kiểu dữ liệu và cú pháp JavaScript
- Xử lý sự kiện với JavaScript
- HTML DOM và JavaScript
- Ví dụ ứng dụng

Giới thiệu JavaScript

- **Client – Side Script**
  - Script được thực thi tại *Client--Side* (trình duyệt): thực hiện các tương tác với người dùng (tạo menu chuyển động, …), kiểm tra dữ liệu nhập,…
- Server – Side Script
  - Script được xử lý tại *Server--Side*, nhằm tạo các trang web có khả năng phát sinh nội dung động. Một số xử lý chính: kết nối CSDL, truy cập hệ thống file trên server, phát sinh nội dung html trả về người dùng…

Giới thiệu JavaScript

- **JavaScript** Là ngôn ngữ *Client--side script* hoạt động trên trình duyệt của người dùng (client)
- *Chia sẻ xử lý* trong ứng dụng web. Giảm các xử lý không cần thiết trên server.
- Giúp *tạo các hiệu ứng, tương tác* cho trang web.

Giới thiệu JavaScript

- Khi trình duyệt *(Client browser)* truy cập trang web có chứa các đoạn mã xử lý tại server-side. Server (run-time engine) sẽ thực hiện các lệnh *Server side Scipts* và trả về nội dung HTML cho trình duyệt.
- Nội dung HTML trả về chủ yếu bao gồm: *mã html, client-script.*

Nhúng JavaScript vào HTML

- *Cách 1. Định nghĩa Script trực tiếp vào trang HTML*

```
<script  type="text/javascript">
<!----
//  Lệnh Javascript
---->
</script>
```

- *Cách 2. Nhúng sử dụng Script cài đặt từ file .js khác*

```
<script src="./abc.js"></script>
```

Nhúng JavaScript vào HTML

- Web Browser sẽ thực thi ***<script>*** khi load trang web theo thứ tự tự *trên xuống* dưới.
- Đối với *Source code JavaScript* có thể đặt trong các file *.js* sẽ được *nhúng* vào file HTML trước khi hoạt động.
- Các đoạn *code JavaScript* được Browser xử *cùng thứ tự* với các thẻ *HTML*. Trừ các ***hàm*** (***function***) chỉ được thực thi khi *có lời gọi hàm*.

Nhúng JavaScript vào HTML

```
<html>
        <head>
                <script  type="text/javascript">
                        some  statements
                </script>
        </head>
        <body>
                <script  type="text/javascript">
                        some  statements
                </script>
                <script  src="Tên_file_script.js">method()</script>
                <script  type="text/javascript">
                        //  gọi thực hiện các phương thức được định nghĩa
                        //  trong "Tên_file_script.js"
                </script>
        </body>
</html>
```

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Demo javascript</title>
    <script type="text/javascript">
        alert("1. Script in head");
    </script>
</head>

<body>
    <script type="text/javascript">
        alert("2. Script in body!");
    </script>

    <script type="text/javascript" src="myScript.js"></script>
    <script type="text/javascript">
        showMessage();
    </script>
</body>
</html>
```


tkhuong@dthu.edu.vn

Demo

- Tạo file html
- Tạo file js

```
<body>
    <script src="test.js"></script>
</body>
```

TestFileJs.html | test.js

test.js

```
1    alert("Hello javascipt with file test.js")
```

Kiểu dữ liệu và cú pháp JavaScript

- Biến số trong JavaScript
- Kiểu dữ liệu trong JavaScript
- Đổi kiểu dữ liệu
- Hàm trong JavaScript
- Các quy tắc chung
- Cú pháp lệnh

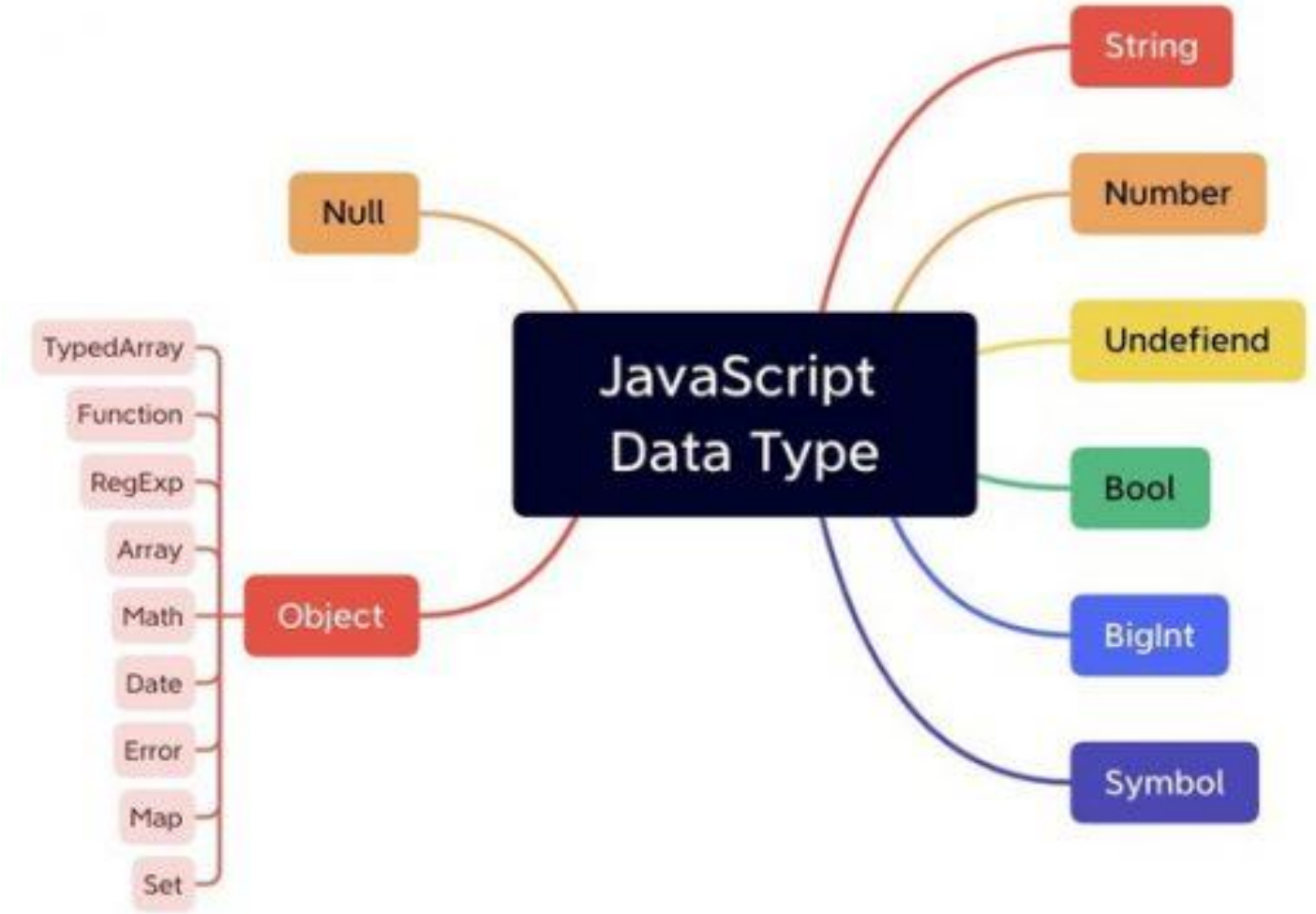
Biến số trong JavaScript

- Cách đặt tên biến
  - Bắt đầu bằng một chữ cái hoặc dấu _
  - A..Z,a..z,0..9,_ : phân biệt HOA, Thường
- Khai báo biến
  - Sử dụng từ khóa **var**
    - Ví dụ: ***var count = 10, amount;***
  - *Không cần khai báo biến trước khi sử dụng*, biến thật sự tồn tại khi bắt đầu sử dụng lần đầu tên.
  - Biến *không cần khải báo kiểu dữ* liệu vì biến trong javascript *không có kiểu dữ liệu nhất định*

Biến số trong JavaScript

- Cách khai báo biến
  - – <const> dùng khai báo hằng số và giá trị bất biến trong suốt chương trình
  - – <let> khai báo biến có thể truy cập được trong các trường bao quanh được xác định bằng cặp {}
  - – <var> dùng khai báo các biến có thể truy cập ở phạm vi hàm số hoặc toàn cục bên ngoài.

Kiểu dữ liệu trong JavaScript



Test

String, Number

Var a =‘tkhuong’;
Var typeName = typeof a; // trả ra choỗi tên kiểu đó
Var typeName = typeof(123);
Console.log(typeName);

Alert(‘hello’)
Var is_confirm = confirm(‘Are you sure?’);
Console.log(is_confirm)

Var name = prompt(‘enter your name’);
Console.log(name)
→ lỗi font thì thêm trên thẻ head: <meta charset=‘utf-8’ />

Kiểu dữ liệu trong JavaScript

Kiểu dữ liệu	Giải thích
Number	- số bất kỳ (int, float,…) - Giá trị đặc biệt: Infinity (vô cực trong toán học); NaN (Not and Number, ko phải số)
String	- dấu + để nối choỗi
Boolean	- True/ False
Null	- giá trị không xác định hoặc không thuộc kiểu dữ liệu
Undefined	- Biến chưa có

Object	- gồm thuộc tính và phương thức; là cặp <khóa – giá trị>
	- Khai báo object: {}; new Object(); static
	Ví dụ:
	// literal const dog = { }
	// constructor const cat = new Object();
	// static method const horse = Object.create({ })
	
	let hello; let world; // Old way const obj = { hello: hello, world: world }
	// Modern way in ES6 const obj = { hello, world, }

Chuyển KDL trong JavaScript

- Biến tự đổi kiểu dữ liệu khi giá trị mà nó lưu trữ thay đổi
  - Ví dụ:
    - Var x = 10; // x kiểu Number
    - X = “hello world!” // x kiểu String
- Có thể cộng 2 biến khác kiểu dữ liệu
  - Ví dụ:
    - Var x;
    - X = “12” + 34.5; // KQ: x=“1234.5”
- Hàm **parseInt(…)**, **parseFloat(…)** : Đổi KDL từ chuỗi sang số.

Hàm trong JavaScript

- Dạng thức khai báo chung

```
function Tên_hàm(thamso1,  thamso2,  …)
{
        …
}
```

- Hàm có giá trị trả về

```
function Tên_hàm(thamso1,  thamso2,  …)
{
        …
        return  (value);
}
```

Hàm trong JavaScript

- Ví dụ:

```
function Sum(x,  y)
{
        tong = x + y;
        return tong;
}
```

- Gọi hàm:

```
Var x = Sum(10, 20);

Alert(x);
```

Một số hàm built-in

- Là hàm mà JavaScript đã xây dựng sẵn
  - Alert
  - Console
  - Confirm
  - Prompt
  - Set timeout
  - Set interval

Các quy tắc chung trong JavaScript

- Khối lệnh được bao trong dấu **{}**
- Mỗi lệnh nên kết thúc bằng dấu **;**
- Cách ghi chú thích:
  - *// Chú thích 1 dòng*
  - */* Chú thích*
    *nhiều dòng ***/***
  - /** */

Câu lệnh IF

```
if (condition)
{
        statement[s] if true
}
else
{
        statement[s] if false
}
```

Ví dụ:

```
var x = 5, y = 6, z;
if (x == 5)
{
        if (y == 6)
                z = 17;
}
else
        z = 20;
```

Thực hành Câu lệnh IF

Hiện thị prompt cho người dùng nhập tên

- nếu user nhấn cancel hoặc không nhập thì hiện thông báo

- nếu đã nhập thì hiện lại tên







Thực hành Câu lệnh IF

Hiện thị confirm cho người dùng xác nhận

- nếu user nhấn ok thì chuyển người sang trang web khác cùng folder

- ngược lại thì hiện thông báo




```
window.location.assign('CreateElement.html');
```

Câu lệnh Switch

```
switch (expression)
{
        case label1 :
                statementlist
        case label2 :
                statementlist
                ...
        default :
                statement list
}
```

Ví dụ:

```
var diem = "G";
switch (diem) {
        case "Y":
                document.write("Yếu");
                break;
        case "TB":
                document.write("Trung bình");
                break;
        case "K":
                document.write("Khá");
                break;
        case "G":
                document.write("Giỏi");
                break;
        default:
                document.write("Xuất sắc");
}
```

Vòng lặp FOR

```
for ([initial expression]; [condition]; [update expression])
{
        statement[s] inside loop
}
```

Ví dụ:

```
var myarray  =  new  Array();
for  (int i = 0; i<10; i++)  {
            myarray[i] = i;

}
```

Vòng lặp WHILE & DO…WHILE

```
while (expression)
{
        statements
}
```

```
do
{
        statements
} while (expression);
```

Ví dụ:

```
var i  =  9, total = 0;
while (i < 10)  {
        total += i * 3 + 5;
        i = i + 5;
}
```

Ví dụ:

```
var i  =  9, total = 0;
do {
        total += i * 3 + 5;
        i = i + 5;
}while (i < 10);
```

Bài tập thực hành

1) Sử dụng lệnh FOR để hiển thị combobox có 31 ngày

2) Viết javascript để lấy ngày hiện tại. Hiển thị dạng:

mm-dd-yyyy, mm / dd / yyyy hoặc dd-mm-yyyy, dd / mm / yyyy

3) Viết chương trình JavaScript để tính tổng của hai số nguyên đã cho. Nếu hai giá trị giống nhau, thì trả về gấp ba tổng của chúng.

4) Viết một chương trình JavaScript để kiểm tra xem một số dương đã cho là bội của 3 hay bội của 7.

5) Viết hàm javascript để in nội dung của trang hiện tại (window.print())

Bài tập thực hành

- Năm nhuận Dương lịch là: năm có chứa 1 ngày dư là 29/2. Cách xác định là năm đó chia hết cho 4, cần lưu ý rằng với những năm tròn thế kỷ (tức số biểu của năm đó có 2 con số không ở cuối) thì phải lấy năm đó chia hết cho 400.
- Năm nhuận âm lịch: năm có 2 tháng bất kỳ nào đó, tức có 13 tháng. Cách xác định là lấy năm đó chia cho 19, nếu số dư là một trong các số: 3, 6, 9, 11, 14, 17

6) Viết javascript để xác định năm được nhập vào có phải là năm nhuận theo dương lịch/ năm nhuận theo âm lịch không?.

Demo JavaScript

```
<script>
var i = 0; // Start point
var images = [];
var time = 1000;
// Image List
images[0] = 'image1.jpg';
images[1] = 'image2.jpg';
images[2] = 'image3.jpg';
images[3] = 'image4.jpg';
// Change Image
function changeImg(){
document.slide.src = images[i];
if(i < images.length - 1){
      i++; } else { i = 0;
}
setTimeout("changeImg()", time);}
window.onload = changeImg;
</script>
```



```
<img name="slide"
width="400" height="200">
```

Xử lý sự kiện với JavaScript

- Các sự kiện được hỗ trợ bởi hầu hết các đối tượng
  - onClick
  - onFocus
  - onChange
  - onBlur
  - onMouseOver
  - onMouseOut
  - onMouseDown
  - onMouseUp
  - onLoad
  - onSubmit
  - onResize
  - …

Xử lý sự kiện cho các thẻ HTML

- Cú pháp:

```
<TAG eventHandler = "JavaScript Code">
```

  - Lưu ý: dấu “…” và ‘…’

```
<div>
        <input type="button"
               name="btnClickMe"
               value="Open new page"
              onclick="window.open('https://www.google.com/');" />
    </div>
```

Xử lý sự kiện bằng FUNCTION

- Ví dụ:

```
<head>
 <script type="text/javascript">
        function ShowMessageFunction() {
            alert("Hello world!");
        }
    </script>
</head>
```

```
<body>
    <div>
            <input type="button" name="btnTest"
                    value="Call function"
                    onclick="ShowMessageFunction();" />
        </div>
</body>
```

Xử lý sự kiện bằng THUỘC TÍNH

- Gán tên hàm xử lý cho 1 object event

```
object.eventhandler = function_name;
```

```
<html>
        <head>
                <script language="Javascript">
                        function GreetingMessage()
                        {
                                window.alert("Welcome to my world");
                        }
                        window.onload = GreetingMessage ()
                </script>
        </head>
        <body>
        </body>
<html>
```

Ví dụ onFocus - onBlur

- Xảy ra khi một thành phần HTML bị focus (onFocus) và mất focus (onBlur)

```
<body>
        <div>
                <form name="frmMain">
                        <input type="text"
                                onfocus="document.bgColor='#cc3366';”
                                onblur="document.bgColor='#66aa33';" />
                </form>
        </div>

        </body>
```

Bài tập thực hành

Viết lại các bài tập trên thành dạng function và gọi sự kiện

HTML DOM và JavaScript

- Đối tượng HTML DOM
- Đối tượng Window – DOM
- Đối tượng Document – DOM

Đối tượng HTML DOM

- **DOM** = Document Object Model
- Là tập hợp các đối tượng HTML chuẩn được dùng để truy xuất và thay đổi thành phần HTML trong trang web (thay đổi nội dung tài liệu của trang)
- Một số đối tượng của DOM: *window, document, history, link, form, frame, locazon, event, …*

Đối tượng Window - DOM

- Là thể hiện của đối tượng cửa sổ trình duyệt
- Tồn tại khi mở 1 tài liệu HTML
- Sử dụng để truy cập thông tin của các đối tượng trên cửa sổ trình duyệt (tên trình duyệt, phiên bản trình duyệt, thanh têu đề, thanh trạng thái …

Đối tượng Window - DOM

- **Properties**
  - **document**
  - event
  - history
  - location
  - name
  - navigator
  - screen
  - status
- **Methods**
  - Alert
  - Confirm
  - Prompt
  - Blur
  - close
  - Focus
  - Open

Đối tượng Window - DOM

- Ví dụ

```
<html>
<body>
        <script  type="text/javascript">
                var curURL  =  window.location;
                window.alert(curURL);
        </script>
</body>
</html>
```

Một số ví dụ window

```
<script type="text/javascript">
        function ShowMessageFunction() {
            alert("Hello world!");
        }

        function fConfirm() {
            var kq = window.confirm("Are you sure?");
            alert(kq);
        }

        function fPrompt() {
            var kq = window.prompt("Nhập vào giá trị: ", 9);
            alert(kq);
        }
    </script>
```

Đối tượng Document - DOM

- Biểu diễn cho nội dung trang HTML đang được hiển thị trên trình duyệt
- Dùng để lấy thông tin về tài liệu, các thành phần HTML và xử lý sự kiện

Đối tượng Document - DOM

- **Properties**
  - aLinkColor
  - bgColor
  - body
  - fgColor
  - linkColor
  - ttle
  - URL
  - vlinkColor
  - forms[]
  - images[]
  - childNodes[]
  - documentElement
  - cookie
  - ……

- **Methods**
  - close
  - open
  - **createTextNode(“ text ")**
  - **createElement(“HTMLtag")**
  - **getElementById(“id”)**
  - …

Đối tượng Document - DOM

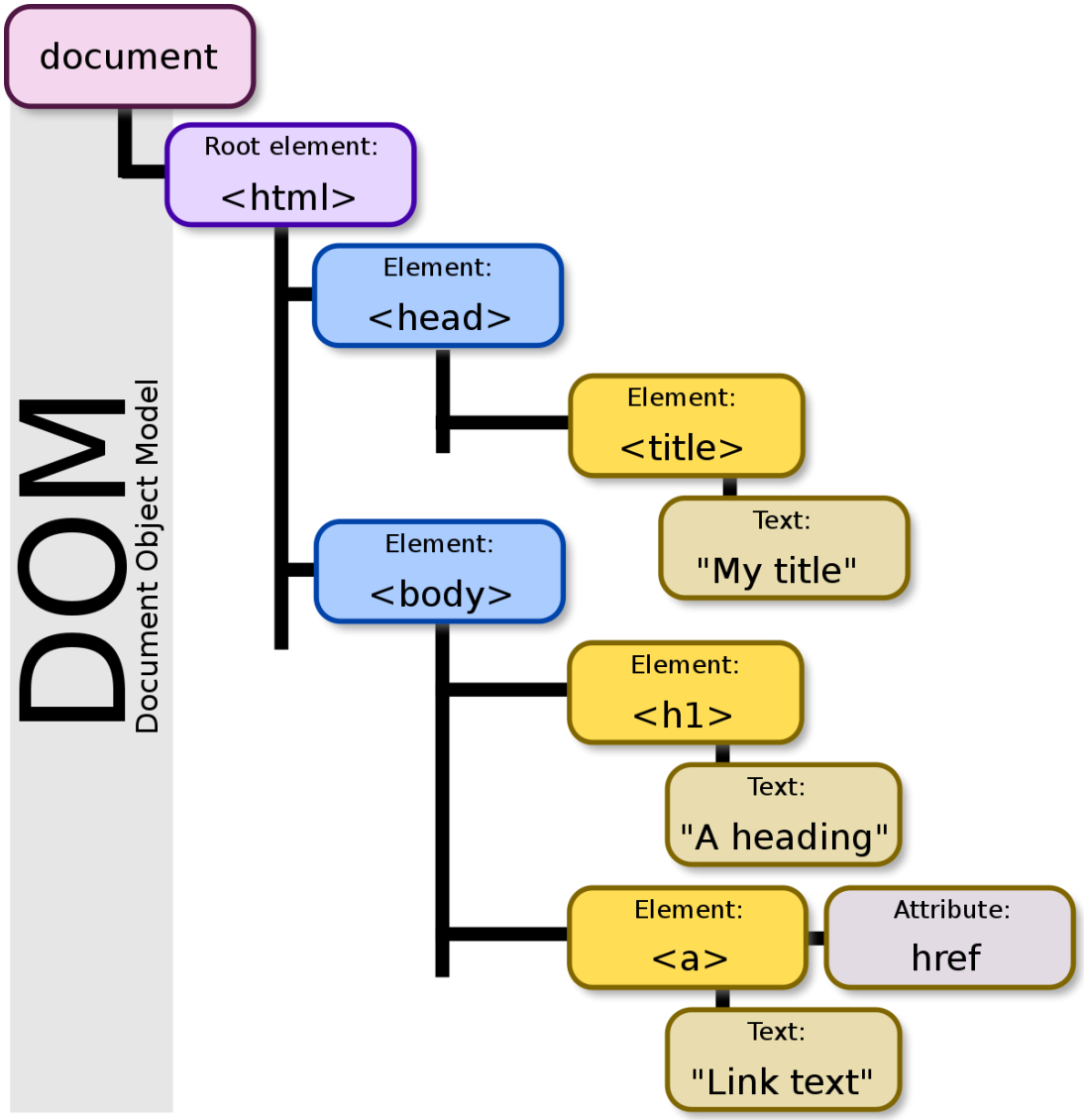
- Biểu diễn nội dung tài liệu theo cấu trúc cây

```
<html>
<head>
        <ttle>DOM  Test</ttle>
</head>
<body>
        <h1>DOM  Test  Heading</h1>
        <hr  />
        <!---- Just  a  comment  ---->
        <p id=“p1” >A  paragraph  of  <em>text</em>  is  just  an  example</p>
        <ul>
                <li>
                        <a  href=“h±p://www.google.com”  >  Google  </a>
                </li>
        </ul>
</body>
</html>
```

Đối tượng Document - DOM

Cây cấu trúc DOM

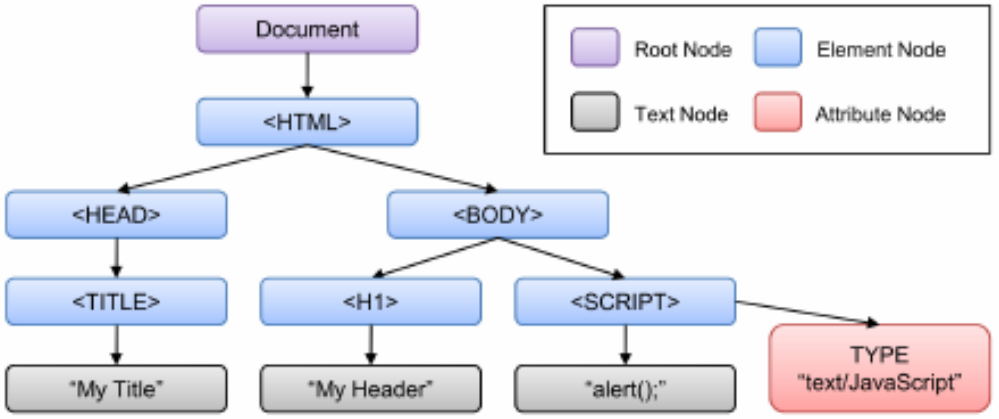
- *Node gốc: là tài liệu HTML, thẻ <html>.*
- *Node phần tử: biểu diễn cho 1 phần tử HTML.*
- *Node văn bản: bên trong 1 thẻ HTML là 1 node văn bản như thẻ <title>, <h1>, <p>.*
- *Ngoài ra còn có node thuộc tính (attribute node) và node chú thích (comment node).*



Đối tượng Document - DOM

Quan hệ giữa các node

- *Node gốc (document) luôn là node đầu tiên.*
- *Tất cả các node không phải là node gốc đều chỉ có 1 node cha (parent).*
- *Một node có thể có một hoặc nhiều con, nhưng cũng có thể không có con nào.*
- *Những node có cùng node cha được gọi là các node anh em (siblings).*
- *Trong các node anh em, node đầu tiên được gọi là con cả (firstChild) và node cuối cùng là con út (lastChild).*



Đối tượng Document - DOM

Các loại DOM trong Javascript

- *DOM document: lưu trữ toàn bộ các thành phần trong documents của website.*
- *DOM element: truy xuất tới thẻ HTML nào đó thông qua các thuộc tính như tên class, id, name của thẻ HTML.*
- *DOM HTML: thay đổi giá trị nội dung và giá trị thuộc tính của các thẻ HTML.*
- *DOM CSS: thay đổi các định dạng CSS của thẻ HTML.*
- *DOM Event: gán các sự kiện như onclick(), onload() vào các thẻ HTML.*
- *DOM Listener: lắng nghe các sự kiện tác động lên thẻ HTML.*
- *DOM Navigation dùng để quản lý, thao tác với các thẻ HTML, thể hiện mối quan hệ cha – con của các thẻ HTML*
- *DOM Node, Nodelist: thao tác với HTML thông qua đối tượng (Object).*

Thao tác với DOM

- Các thuộc tính

Thuộc tính	Ý nghĩa
Id	Định danh – là duy nhất cho mỗi phần tử nên thường được dùng để truy xuất DOM trực tiếp và nhanh chóng
className	Tên lớp – Cũng dùng để truy xuất trực tiếp như id, nhưng 1 className có thể dùng cho nhiều phần tử.
tagName	Tên thẻ HTML.
innerHTML	Trả về mã HTML bên trong phần tử hiện tại. Đoạn mã HTML này là chuỗi kí tự chứa tất cả phần tử bên trong, bao gồm các node phần tử và node văn bản.
outerHTML	Trả về mã HTML của phần tử hiện tại. Nói cách khác, outerHTML = tagName + innerHTML.
textContent	Trả về 1 chuỗi kí tự chứa nội dung của tất cả node văn bản bên trong phần tử hiện tại.
Attributes	Tập các thuộc tính như id, name, class, href, title…
Style	Tập các định dạng của phần tử hiện tại
value	Lấy giá trị của thành phần được chọn thành một biến.

Thao tác với DOM

- Các phượng thức

Phương thức	Ý nghĩa
getElementById(id)	Tham chiếu đến 1 node duy nhất có thuộc tính id giống với id cần tìm.
getElementsByTagName(tagname)	Tham chiếu đến tất cả các node có thuộc tính tagName giống với tên thẻ cần tìm. muốn truy xuất đến toàn bộ thẻ trong tài liệu HTML thì hãy sử dụng document.getElementsByTagName('*')
getElementsByName(name)	Tham chiếu đến tất cả các node có thuộc tính name cần tìm.
getAttribute(attributeName)	Lấy giá trị của thuộc tính.
setAttribute(attributeName, value)	Sửa giá trị của thuộc tính.
appendChild(node)	Thêm 1 node con vào node hiện tại.
removeChild(node)	Xóa 1 node con khỏi node hiện tại.

Thao tác với DOM

- Thuộc tính quan hệ
  - parentNode: node cha
  - childNodes: Các node con
  - firstChild: node con đầu tiên
  - lastChild: node con cuối cùng
  - nextSibling: node anh em liền kề sau
  - previousSibling: node anh em liền kề trước

Thao tác với DOM

- Truy xuất các phần tử trong mô hình DOM
  - document.getElementById()
  - document.getElementsByTagName()
  - document.getElementsByClass()

- **Note**
  - *document.getElementById(): giá trị trả về chính là phần tử có id đó.*
  - *document.getElementsByTagName() hoặc document.getElementsByClass(): giá trị trả về sẽ là một mảng, vậy nên ta cần truy xuất phần tử của mảng để xác định được phần tử mong muốn.*

Ví dụ DOM

Truy xuất và thay đổi nội dung của thẻ h1 bằng phương thức document.getElementById()

```
<body>
        <h1 id="txt1">Truy xuất bằng id</h1>
        <script type="text/javascript">
                var x = document.getElementById('txt1’);
                // Truy xuất thẻ h1 có id 'txt1’
                x.innerHTML = "Thay đổi";
                //Thay đổi nội dung của thẻ h1
        </script>
</body>
```

Ví dụ DOM

Truy xuất và thay đổi nội dung của thẻ h1 bằng phương thức document.getElementTagName()

```
<body>
    <h1 id="txt1">Truy xuất bằng TagName</h1>

    <script type="text/javascript">
        var x = document.getElementsByTagName('h1');
        //Truy xuất các thẻ có TagName là h1
        x[0].innerHTML = 'Thay đổi';
        //Thay đổi nội dung của thẻ h1 đầu tiên trong html
    </script>

</body>
```

Đối tượng Document - DOM

- getElementById (*id1*)
  - Trả về node có giá trị thuộc tính id = *id1*
  - *Ví dụ:*

```
//<p id="id1">
// some text
//</p>
Var node = document.getElementById("id1");
Var nodeName = node.nodeName; //p
Var nodeType = node.nodeype; //1
Var nodeValue = node.nodeValue; // null
Var text = node.innerText; // some text
```

Đối tượng Document - DOM

- createElementById (*nodeName*)
  - Cho phép tạo ra 1 node HTML mới tùy theo đối số *nodeName* đầu vào
  - *Ví dụ:*

```
var imgNode = document.createElement("img");
imgNode.src = "images/text.png";

// <image src="images/test.png />
```

Đối tượng Document - DOM

- createTextNode (*content*)
  - Cho phép tạo ra 1 node HTML mới tùy theo đối số *nodeName* đầu vào
  - *Ví dụ:*

```
var textNode = document.createTextNode("New text");
var pNode = document.createElement("p");
pNode.appendChild(textNode);
// <p>New text</p>
```

Đối tượng Document - DOM

- appendChild (*newNode*)
  - Chèn node mới *newNode* vào cuối danh sách các node con của một node
  - *Ví dụ:*

```
//<p  id="id1"  >
// some  text
//</p>
var pNode =  document.getElementById(“id1”);
var imgNode =  document.createElement(“img”);
imgNode.src  =  “images/test.gif”;
pNode.appendChild(imgNode);

//<p  id="id1"  > some  text  <img src=“images/test.gif”  />  /</p>
```

Demo document Node

```
<script type="text/javascript">
        function addImage() {
            var posNode = document.getElementById("pos"); // ánh
xạ thẻ div vào node
            var imgNode = document.createElement("img");
            imgNode.src = "images/sentuyet.jpg";

            posNode.appendChild(imgNode);
        }
    </script>
```

```
<body>
    <input type="button" value="Add image"
onclick="addImage()" />
    <div id="pos"></div>
</body>
```

Đối tượng Document - DOM

- innerHTML
  - Chỉ định nội dung HTML bên trong một node
  - *Ví dụ:*

```
//<p  id=“para1"  >
// some  text
//</p>
var theElement  =  document.getElementById("para1");
theElement.innerHTML  =  “Some  <b>  new  </b>  text”;

//  Kết quả  :
//  <p  id=“para1”  >
//  Some  <b>  new  <b/>  text
//  </p>
```

Đối tượng Document - DOM

- innerText
  - Tương tự HTML, tuy nhiên bất kỳ nội dung nào đưa vào cũng được xem là text hơn là các thẻ HTML
  - *Ví dụ:*

```
var theElement = document.getElementById("para1");
theElement.innerText = “Some <b> new </b> text”;

// Kết quả hiển thị trên trình duyệt
// bên trong thẻ p: “Some <b> new </b> text”
```

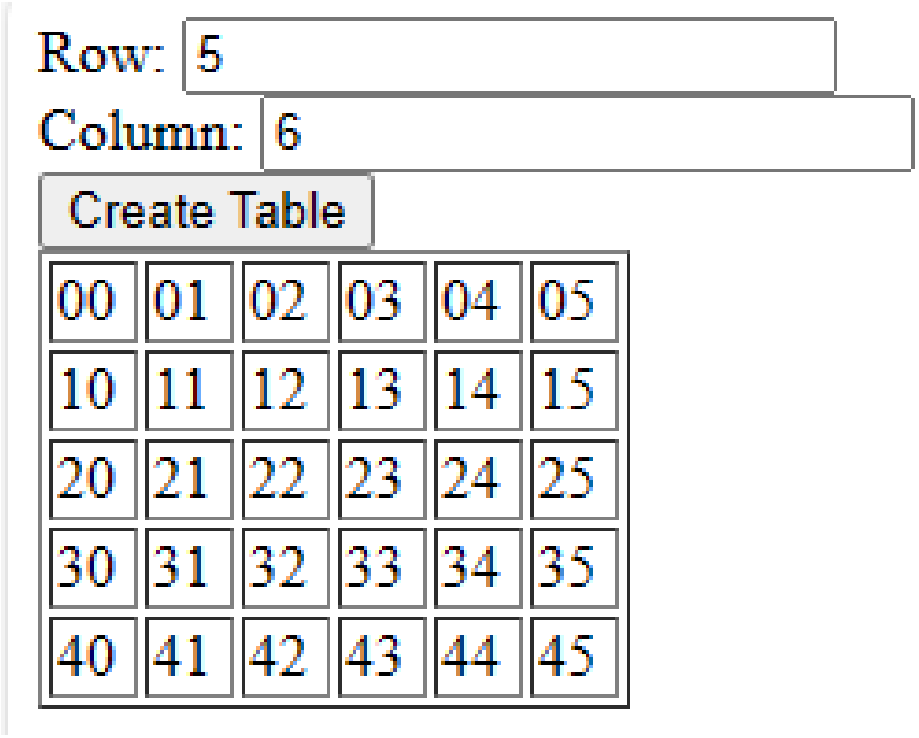
Demo innerHTML/ innerText

```
function changeHTML(){
            var divNode =
document.getElementById("pos1");
            divNode.innerHTML = "Chào <b>các bạn
</b>";
        }
```

```
 <input type="button" value="Change HTML"
       onclick="changeHTML()" />
    <div id="pos1"></div>
```

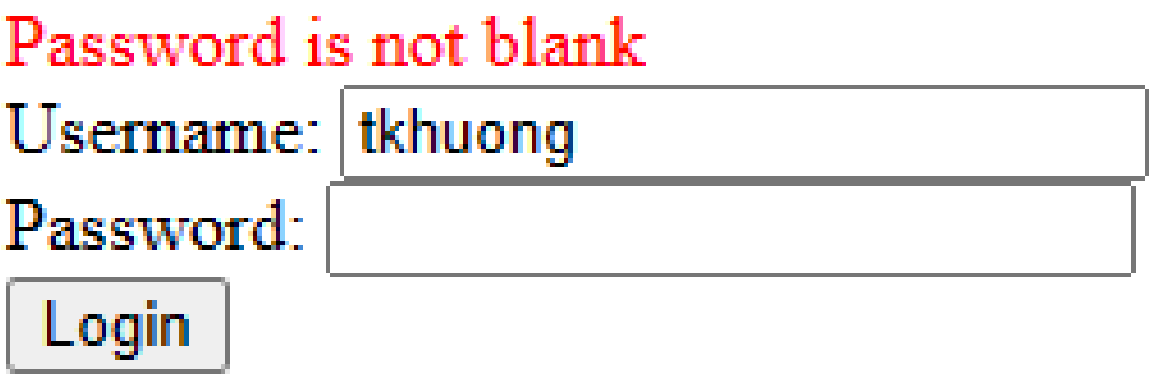
Bài tập thực hành 1

1) Viết trang web cho phép tạo table có số dòng, số cột do người dùng nhập vào



Bài tập thực hành 2

2) Kiểm tra dữ liệu nhập trước cho form Đăng nhập khi Submit



Bài tập thực hành

1) Viết hàm javascript thực hiện phép tính nhân số x nhập vào từ bàn phím: x*x

1) Viết chương trình javascript để biểu diễn Calculator



DISCUSSION!

tkhuong@dthu.edu.vn